

Tape merge is optimal up to a size ratio of $148/95$

Yunjiang Jiang

3 October 2026

Abstract

Let $M(m, n)$ be the minimum worst-case number of pairwise comparisons needed to merge two sorted lists of distinct elements, of lengths m and n . We give a computer-assisted proof that $M(m, n) = m + n - 1$ whenever $1 \leq m \leq n \leq (148/95)m$. The constant $148/95 = 1.5578947368 \dots$ improves on the $38/25 = 1.52$ bound of Li, Sun and Zhang. We retain Knuth's disjunctive adversary and extend the block-translation argument of Li, Sun and Zhang. A parameterized induction reduces the all-size statement to 138,622 integer inequalities, seven boundary values, and 10,651 finite seed pairs. A two-direction block decomposition removes the need for monotonicity lemmas in the final passage to all list sizes. Two separately implemented recurrences agree on every entry of the required finite table. We also give independently checked decision-tree certificates for $M(6, 22) = 21$ and $M(6, 25) = 22$, values conjectured by Smith and Lang in 1994. A secondary corollary of published upper bounds shows $M(m, 2m - 3) \leq 3m - 5$ for $m \geq 9$. All recurrences, exceptional cases, and verification procedures are specified.

Contents

1	Introduction	2
1.1	A simple algorithm and a difficult optimality question	2
1.2	Historical context and scope	3
1.3	Results and attribution	3
1.4	How the main proof works	4
2	A recursive adversary and its meaning	5
3	A self-contained finite-to-infinite translation argument	9
3.1	Four auxiliary facts, with proofs	9
3.2	The seven boundary checks and the finite translation square	10
3.3	Copying a split and locating every exceptional child	11
3.4	Proof of the six global translations	12
3.5	A finite cone of residual sizes	15
3.6	The concrete parameters and the finite obligations	16
4	The finite computation and reproduction	16
4.1	Evaluating the recurrence exactly	16
4.2	The checked hypotheses	17
4.3	What is and is not certified	17

5	An unrestricted finite certificate for exact merging	18
5.1	Exact states and their legal queries	18
5.2	Ordinal-sum decomposition	19
5.3	Elementary bounds with complete recurrences	20
5.4	A finite, well-founded certificate	21
5.5	The two checked instances	22
6	Further directions	23
A	An upper-bound consequence of the modified binary merge	23
B	Reproducing the finite proof	26

1 Introduction

1.1 A simple algorithm and a difficult optimality question

Suppose that

$$a_1 < a_2 < \cdots < a_m, \quad b_1 < b_2 < \cdots < b_n$$

are already sorted. All $m + n$ elements are distinct. Their values can be accessed only through comparisons: a query asks whether $a_i < b_j$ or $a_i > b_j$. Comparisons within either list give information already known, so only comparisons between lists are useful. The task is to determine the single sorted order of all the elements.

An algorithm may choose each query using the answers to earlier queries. Its worst-case cost is the greatest number of queries it makes on any possible interleaving of the two lists. We consider deterministic algorithms and count comparisons, without imposing a bound on other computation or memory. The smallest possible worst-case cost is denoted by $M(m, n)$. By exchanging the lists, $M(m, n) = M(n, m)$; we usually take $m \leq n$. If one list is empty, no comparison is needed, so $M(m, 0) = M(0, n) = 0$.

Such an algorithm can be drawn as a binary decision tree: an internal node specifies a comparison, its two branches are the possible answers, and a leaf specifies the final merged order. An execution's depth is its number of comparisons; the tree's height is the largest such depth. We write $\lfloor x \rfloor$ and $\lceil x \rceil$ for the largest integer at most x and the smallest integer at least x , respectively. All logarithms have base two.

The familiar *tape merge* repeatedly compares the first remaining element of each list and outputs the smaller one. Each comparison removes one element. Once one list is empty the other can be appended, so

$$M(m, n) \leq m + n - 1 \quad (m, n \geq 1). \tag{1}$$

Its worst case attains this bound. Optimality asks whether *every* correct algorithm has some input on which it needs this many comparisons. For very unequal list lengths the answer is no: for $m = 1$, binary search in the $n + 1$ possible insertion positions gives $M(1, n) = \lceil \log_2(n + 1) \rceil$.

There are exactly $\binom{m+n}{m}$ possible interleavings, because choosing the m positions occupied by the a -elements determines the entire order. A binary decision tree of height h has at most 2^h leaves; distinct interleavings need distinct leaves. Consequently

$$\left\lceil \log_2 \binom{m+n}{m} \right\rceil \leq M(m, n). \tag{2}$$

This counting argument alone generally leaves a gap. Queries are restricted to comparing two elements; they cannot ask an arbitrary yes/no question that divides the remaining interleavings in half. The issue is therefore both how many possibilities remain and how comparisons can separate them.

1.2 Historical context and scope

Graham and Karp independently established the optimality of tape merge for equal lengths; Knuth records this result and develops the adversary method used here [7, Section 5.3.2]. Hwang and Lin introduced binary merge, which adapts to unequal list lengths [6]; they also determined $M(2, n)$ [5]. The cases of three and four elements in the smaller list required further work by Hwang, Murphy, and Mönting [4, 12, 14]. Mönting also obtained substantial results for five elements.

The first broad tape-optimality region was

$$m \leq n \leq \left\lfloor \frac{3}{2}m \right\rfloor + 1.$$

It was proved independently by Stockmeyer and Yao [16], Murphy and Paull [13], and Christen; the last result is cited as an unpublished 1978 manuscript by Li, Sun and Zhang [2, 9]. For lists farther from equal size, Christen’s forward-testing and backward-insertion method and subsequent improvements to binary merge provide different upper bounds [1, 11, 17]. These algorithmic results are relevant background, but their procedures are not used in the proof of our main theorem.

Smith and Lang’s 1994 report [15] used exact game search and computable adversary recurrences to obtain many finite values and bounds. One striking value is $M(7, 12) = 17$: tape merge would use 18 comparisons. Their work also makes clear why a large numerical table is not, by itself, a theorem about all list sizes. Li, Sun and Zhang [8, 9] crossed the asymptotic ratio $3/2$ by proving tape optimality for $n \leq (38/25)m$. Their proof translates a finite adversary block of size $(25, 38)$ into arbitrarily large instances. They also prove that the same disjunctive adversary cannot certify tape optimality when $n \geq 9 \lceil m/5 \rceil$, and give improved algorithms near $n = 2m$.

Other merging models have different objectives. The randomized expected comparison bounds of Fernández de la Vega, Kannan and Santha [3] do not give deterministic worst-case bounds. Likewise, parallel depth, in-place storage, and the total running time of a mergesort are different questions from the function M considered here. Linial’s result on merging under a known partial order [10] is particularly relevant to our finite exact computation, although that theorem is not needed by our verifier.

1.3 Results and attribution

Theorem 1.1. *For all positive integers m, n ,*

$$m \leq n \leq \frac{148}{95}m \implies M(m, n) = m + n - 1.$$

The gain over $38/25$ is $18/475$ in the ratio. To our knowledge, no stronger general ratio bound has been published; the literature review for this manuscript was completed on 3 October 2026. For example, at $m = 1000$ the new bound reaches $n = 1557$, whereas $38/25$ reaches $n = 1520$. This is an extension of the Li–Sun–Zhang method within Knuth’s existing adversary framework. It does not remove the framework’s separate obstruction near ratio 1.8. The earlier additive region $n \leq \lfloor 3m/2 \rfloor + 1$ may still be stronger for some small m ; the two results can be combined by taking their union.

Theorem 1.2.

$$M(6, 22) = 21, \quad M(6, 25) = 22.$$

Smith and Lang already proved the corresponding upper bounds and explicitly conjectured both equalities [15, Sections 3.12 and 4.3]. Our certificates verify unrestricted lower bounds as well as matching upper bounds. These assertions concern all legal comparison algorithms, not only the bounded-width search classes considered in part of their report.

Proposition 1.3. *For every integer $m \geq 9$, $M(m, 2m - 3) \leq 3m - 5$. Consequently tape merge is suboptimal at every $n \geq 2m - 3$.*

This is a short consequence of Smith–Lang finite algorithms and the Li–Sun–Zhang recurrence and upper estimates. We include it as an explicit corollary, not as a new merging procedure or a matching lower bound. It supplies the upper-bound half of a family conjectured by Smith and Lang [15, Section 5.1].

1.4 How the main proof works

The proof has a finite part and an infinite part, connected by an explicit induction. The following outline explains why both parts are needed.

1. **Make the adversary recursive.** An adversary answers comparisons so as to delay the algorithm, while keeping at least one input consistent with every answer. We restrict it to answers that separate the remaining task into two smaller tasks. Some separations share a single extreme element. Keeping track of which end elements are privately constrained leads to nine recursively defined quantities $D_{\lambda\rho}(m, n)$. Their unconstrained member satisfies $D_{00}(m, n) \leq M(m, n)$.
2. **Find a block that pays for all its elements.** For $A = 95$, $B = 148$, and $S = A + B = 243$, we prove translations of the form

$$D_{\lambda\rho}(m + A, n + B) \geq D_{\lambda\rho}(m, n) + S.$$

This is stronger than simply putting two hard instances side by side: that operation would ordinarily lose a comparison at each boundary. We must instead answer *every possible first comparison*, including comparisons that cross the proposed block boundary.

3. **Repair the exceptional cases.** Six boundary types suffice by symmetry. Two tiny states do not obey the translation. Endpoint-deletion cuts and a uniform analysis of the comparison $a_2 : b_1$ repair every place where one of them appears as a child. Children containing elements from only one list are handled separately. This gives an induction from a finite square to all dimensions.
4. **Fill the whole ratio region.** Exchanging the lists gives translations by both (A, B) and (B, A) . Any integer pair in the cone between these directions is a nonnegative integer combination of the two blocks plus a positive residual pair with both coordinates below S . Verifying the residual pairs completes the proof. This last step uses no monotonicity theorem or conjecture.

The computer checks only finite integer assertions. The all-size induction, its exceptional cases, and the residual-pair argument are proved below. The main finite table is computed twice, by differently organized implementations of the same explicitly stated recurrence. The exact certificates for Theorem 1.2 use a separate unrestricted state space and a standalone verifier. The algorithms, accepted certificates, and reproduction instructions accompany the source of this article.

2 A recursive adversary and its meaning

All keys are distinct. The input consists of two chains $a_1 < \dots < a_m$ and $b_1 < \dots < b_n$. A merging decision tree must be correct for every interleaving of these chains. Let $M(m, n)$ be the minimum worst-case number of comparisons in such a tree. Comparisons within a chain carry no information and may be omitted.

Private restrictions are not public promises. We refer to the two list labels as colors: a block is monochromatic if it contains keys from only one list, and bichromatic if it contains keys from both. Write $0, L, R$ for Knuth's dot, backslash, and slash; the symbols λ and ρ take values in this three-element set. For $m, n > 0$ a left status L restricts the adversary to completions with $a_1 < b_1$, and a left status R restricts it to completions with $a_1 > b_1$. At the right, L means $a_m < b_n$ and R means $a_m > b_n$. Status 0 imposes no restriction. These restrictions are obligations on the adversary; they are not comparison outcomes given to the algorithm. An algorithm can use an endpoint inequality once the record of public comparison answers implies it, but cannot introduce it as a premise merely because the adversary's private state carries that status.

Here is a precise way to distinguish these two notions. Let $\mathcal{W}$ be the set of all chain interleavings, and let $\mathcal{W}_{\lambda\rho}$ be its subset satisfying the two endpoint restrictions. Define

$$H_{\lambda\rho}(m, n) = \min_{T \text{ correct on } \mathcal{W}} \max_{w \in \mathcal{W}_{\lambda\rho}} \text{depth}_T(w).$$

The minimum is still over globally correct trees, even though the objective only uses the restricted subset. A public-promise problem would instead require correctness only on that subset, and is different. For example, $H_{LL}(1, 1) = 1$, whereas the corresponding public-promise problem needs no comparison. Likewise, the private state $LR(2, 1)$ has only the permitted order $a_1 < b_1 < a_2$, but both adjacent cross-chain relations must be established publicly, and its cost is two.

We define $D_{\lambda\rho}$ below by the recursive disjunctive rule. The proof establishes

$$D_{\lambda\rho}(m, n) \leq H_{\lambda\rho}(m, n) \leq M(m, n), \quad D_{00}(m, n) \leq M(m, n). \quad (3)$$

Thus no identification of D with the unrestricted hidden-constraint minimax problem is required. In particular, the recurrence defines the value of this recursive counterstrategy, not an assertion that every possible private adversary is captured by it.

Feasibility and terminal conventions. For positive dimensions, the only impossible endpoint states are

$$m = 1, (\lambda, \rho) = (L, R), \quad n = 1, (\lambda, \rho) = (R, L). \quad (4)$$

Indeed, left L and right R require the first and last elements of the merged order to be a-elements, and left R and right L require them both to be b-elements. These requirements are feasible exactly when the corresponding chain has at least two elements; all remaining endpoint combinations are feasible. Assign impossible states value $-\infty$ and use $-\infty + x = -\infty$. They are never zero-cost alternatives.

A one-chain problem is terminal and costs zero. Its nonexistent endpoint comparisons are not retained as unresolved restrictions. When a private cut produces an a-only prefix, the parent's left status must be 0 or L ; for a b-only prefix it must be 0 or R . An a-only suffix permits right status 0 or R ; a b-only suffix permits 0 or L . Compatible restrictions are then dropped. This is the legal normalization of monochromatic children.

For writing the six published formulas compactly, introduce a padded value $\widehat{D}$ equal to D at positive dimensions and satisfying

$$\widehat{D}_{\lambda\rho}(m, 0) = \widehat{D}_{\lambda\rho}(0, n) = 0 \quad \text{for every } \lambda, \rho.$$

These padded entries are used only with the endpoint exclusion tests below. They do not make arbitrary constrained zero-axis states legal games. In particular, the unambiguous unconstrained convention is $D_{00}(m, 0) = D_{00}(0, n) = 0$. A feasible state with $m, n > 0$ is not terminal, even when its private restrictions select a unique interleaving.

Six responses to a query. Fix a feasible parent (m, n, λ, ρ) with $m, n > 0$, and a first comparison $a_i : b_j$. Intervals such as $a_{u:v}$ denote the corresponding consecutive subsequence, empty when $u > v$. In the following table the statuses beside a child are its left and right private statuses. The unprimed rows answer $a_i < b_j$; the primed rows answer $a_i > b_j$.

type	left child	right child	shared key
A	$(a_{1:k}, b_{1:\ell-1}); (\lambda, 0)$	$(a_{k+1:m}, b_{\ell:n}); (0, \rho)$	none
B	$(a_{1:k}, b_{1:\ell}); (\lambda, L)$	$(a_{k+1:m}, b_{\ell:n}); (R, \rho)$	b_ℓ
C	$(a_{1:k}, b_{1:\ell-1}); (\lambda, R)$	$(a_{k:m}, b_{\ell:n}); (L, \rho)$	a_k
A'	$(a_{1:k-1}, b_{1:\ell}); (\lambda, 0)$	$(a_{k:m}, b_{\ell+1:n}); (0, \rho)$	none
B'	$(a_{1:k-1}, b_{1:\ell}); (\lambda, L)$	$(a_{k:m}, b_{\ell:n}); (R, \rho)$	b_ℓ
C'	$(a_{1:k}, b_{1:\ell}); (\lambda, R)$	$(a_{k:m}, b_{\ell+1:n}); (L, \rho)$	a_k

Each child is reindexed starting at one. All left-only keys are privately placed before all right-only keys. For an overlap, the repeated key is privately the maximum of the left child and the minimum of the right child. The displayed inner statuses impose exactly these requirements: an a-pivot has right status R on the left and left status L on the right; a b-pivot has right status L and left status R .

The complete parameter and endpoint restrictions are

type	range of k	range of ℓ	excluded endpoints
A	$i \leq k \leq m$	$1 \leq \ell \leq j$	$\ell = 1, \lambda = R$ or $k = m, \rho = R$
B	$i \leq k \leq m$	$1 \leq \ell < j$	$\ell = 1, \lambda = R$ or $k = m, \rho = R$
C	$i < k \leq m$	$1 \leq \ell \leq j$	$\ell = 1, \lambda = R$ or $k = m, \rho = R$
A'	$1 \leq k \leq i$	$j \leq \ell \leq n$	$k = 1, \lambda = L$ or $\ell = n, \rho = L$
B'	$1 \leq k \leq i$	$j < \ell \leq n$	$k = 1, \lambda = L$ or $\ell = n, \rho = L$
C'	$1 \leq k < i$	$j \leq \ell \leq n$	$k = 1, \lambda = L$ or $\ell = n, \rho = L$

In addition, any positive-dimensional child forbidden by (4) invalidates the entire response. The strict bounds in rows B, C, B', C' are essential: the queried keys must not belong together to a child, and neither may be the repeated pivot. They also prevent a child from having the parent's total size. Every child of an admissible row has strictly fewer than $m + n$ keys, counting the pivot once in each child that contains it.

The corresponding continuation scores, in the same order, are

$$\begin{aligned}
A(k, \ell) &= \widehat{D}_{\lambda 0}(k, \ell - 1) + \widehat{D}_{0\rho}(m - k, n + 1 - \ell), \\
B(k, \ell) &= \widehat{D}_{\lambda L}(k, \ell) + \widehat{D}_{R\rho}(m - k, n + 1 - \ell), \\
C(k, \ell) &= \widehat{D}_{\lambda R}(k, \ell - 1) + \widehat{D}_{L\rho}(m + 1 - k, n + 1 - \ell), \\
A'(k, \ell) &= \widehat{D}_{\lambda 0}(k - 1, \ell) + \widehat{D}_{0\rho}(m + 1 - k, n - \ell), \\
B'(k, \ell) &= \widehat{D}_{\lambda L}(k - 1, \ell) + \widehat{D}_{R\rho}(m + 1 - k, n + 1 - \ell), \\
C'(k, \ell) &= \widehat{D}_{\lambda R}(k, \ell) + \widehat{D}_{L\rho}(m + 1 - k, n - \ell).
\end{aligned}$$

These are Smith–Lang [15], equations (22)–(27), with $0, L, R$ substituted for their symbols. Let $\mathcal{R}_{\lambda\rho}(m, n; i, j)$ be the legal rows and parameters specified above. The exact recurrence being computed is

$$D_{\lambda\rho}(m, n) = 1 + \min_{1 \leq i \leq m, 1 \leq j \leq n} \max_{r \in \mathcal{R}_{\lambda\rho}(m, n; i, j)} \text{score}(r). \quad (5)$$

For a feasible parent and any query, the response set is nonempty. To see this, choose any feasible full interleaving and put a simple cut between the queried keys. The resulting consecutive prefix and suffix obey the required endpoint tests. Thus the maximum is at least zero. Replacing the zeroed exclusion terms of Smith–Lang’s equation (28) by discarded terms, or adding an explicit zero to this maximum, does not change the value. This observation does not permit an impossible positive-dimensional child to contribute zero to an otherwise valid sum.

Why the two children can be charged independently. Splits and endpoint statuses remain private. Subsequent comparisons whose two keys occur in one child are delegated to that child. Such a child is unique: the children are either disjoint or share only one key, whereas a comparison has two distinct keys. Every other cross-chain comparison is between a left-only and a right-only key and receives the fixed left-before-right answer. It is an additional paid comparison, not a free disclosure of the private split.

There are two facts to prove. First, independent child answers are jointly consistent. For a simple split, concatenate endpoint-compatible child completions. For an overlap, the pivot is last in the left completion and first in the right completion; concatenate and identify the two copies. This gives an interleaving of the original chains, satisfies the parent’s endpoint statuses, and has the advertised answer to the first query. The pivot is one physical key, not a duplicated independent input.

Second, fixed cross-child answers cannot replace an unresolved internal comparison certificate. Consider any alternative completion of one child consistent with that child’s public local transcript. This alternative need not obey its private endpoint statuses. Keep a completion of the sibling that does obey its statuses. For a simple split, concatenate the two orders. For a pivot split, if the altered child is the left one, concatenate its entire alternative order with the right order after deleting the right order’s first element, the pivot. If the altered child is the right one, concatenate the left order after deleting its last element, the pivot, with the entire alternative right order. In both constructions the original a-chain and b-chain remain in order, the sibling’s order is preserved, and every left-only key is still before every right-only key. All delegated and cross-child comparison answers therefore remain true. Comparisons involving the pivot and another child key were delegated to that child and are already included in its local transcript.

Consequently, if one child’s local transcript allows two different interleavings, the full transcript also allows two different interleavings. The altered full interleaving may violate a private restriction, which is immaterial: the merging algorithm is required to be correct on that interleaving too. This replacement argument composes up an arbitrary recursive split tree. At each ancestor the sibling can be kept at an endpoint-compatible completion, so all previously answered cross comparisons are preserved. Revealing the split together with its order inequalities as a public promise would destroy this argument and would define the wrong game.

Proof of the lower-bound interpretation. Induct on $m + n$. The zero-axis cases are immediate. At a feasible positive state the public transcript initially contains only the two chain orders, so a globally correct algorithm must make a cross-chain comparison. For its first query, the adversary chooses a legal response maximizing the score in (5) and then runs the two inductively supplied child strategies. All actual completions obey the private statuses by the consistency argument.

The replacement argument forces the algorithm to finish a complete public certificate in each child before it can terminate. The induction hypothesis charges at least the first child's D -value to comparisons delegated there and at least the second child's D -value to comparisons delegated there. No comparison is charged twice, even for an overlapping split. The first query belongs to neither child and costs one additional comparison. Any later cross-child comparisons only increase the total. The chosen response therefore forces its score plus one. Taking the minimum over possible first queries proves $D_{\lambda\rho} \leq H_{\lambda\rho}$ for every feasible state, and (3) follows. Tape merge supplies $M(m, n) \leq m + n - 1$ for positive m, n .

Three geometric split forms and the primary implementation. The six rows can equivalently be organized into three forms, each with two possible query rectangles:

$$\begin{aligned} \text{simple: } & (p, q), (m - p, n - q), \quad i \leq p, j \geq q + 1 \quad \text{or} \quad i \geq p + 1, j \leq q; \\ \text{a-pivot: } & (p, q), (m - p + 1, n - q), \quad i \leq p - 1, j \geq q + 1 \quad \text{or} \quad i \geq p + 1, j \leq q; \\ \text{b-pivot: } & (p, q), (m - p, n - q + 1), \quad i \leq p, j \geq q + 1 \quad \text{or} \quad i \geq p + 1, j \leq q - 1. \end{aligned}$$

The simple form uses $0 \leq p \leq m$, $0 \leq q \leq n$ and excludes an empty or full prefix. Its statuses are $(\lambda, 0)$, $(0, \rho)$, normalized on monochromatic children as above. The genuine a-pivot form uses $1 \leq p \leq m$, $1 \leq q < n$ and statuses (λ, R) , (L, ρ) . The genuine b-pivot form uses $1 \leq p < m$, $1 \leq q \leq n$ and statuses (λ, L) , (R, ρ) .

The substitutions linking the two organizations are

$$\begin{aligned} A : (p, q) &= (k, \ell - 1), & A' : (p, q) &= (k - 1, \ell), \\ B : (p, q) &= (k, \ell), & B' : (p, q) &= (k - 1, \ell), \\ C : (p, q) &= (k, \ell - 1), & C' : (p, q) &= (k, \ell). \end{aligned}$$

The published padded formulas also include four degenerate pivot cases. They are sound but unnecessary. Dropping a private endpoint restriction cannot decrease D : this follows directly by induction in the recurrence, since every former response is still available and its affected child value can only increase. Therefore each degenerate case is dominated by the following simple cut:

degenerate row	monochromatic child	dominating simple cut (p, q)
$B, k = m$	right b-only	(m, ℓ)
$C, \ell = 1$	left a-only	$(k - 1, 0)$
$B', k = 1$	left b-only	$(0, \ell - 1)$
$C', \ell = n$	right a-only	(k, n)

In each row the nonzero child retains its dimensions and loses one private endpoint restriction. The table's strict query bounds make the dominating simple cut applicable, and the endpoint exclusions make it compatible with the parent. This proves the equivalence between the literal six-formula program and the primary three-form program, including their different zero-axis representations.

Symmetries. Define $\bar{0} = 0$, $\bar{L} = R$, $\bar{R} = L$. Swapping the two lists, and reversing the entire order while reversing each list, respectively give

$$D_{\lambda\rho}(m, n) = D_{\bar{\lambda}\bar{\rho}}(n, m), \quad D_{\lambda\rho}(m, n) = D_{\bar{\rho}\bar{\lambda}}(m, n).$$

Both maps preserve feasibility, normalized terminal cases, query rectangles, and the complete set of legal response trees, so these are exact equalities for the recurrence. In particular, $D_{0L} = D_{R0}$, $D_{0R} = D_{L0}$ and $D_{LL} = D_{RR}$ at fixed dimensions. The reversal-fixed states are $00, LR, RL$; a convenient set of six representatives is $00, L0, R0, LL, LR, RL$. The list-swap symmetry additionally allows the computation to be restricted to $m \leq n$.

3 A self-contained finite-to-infinite translation argument

The purpose of this section is to explain why a finite exact computation can prove an assertion for lists of every length. The computation checks a finite collection of inequalities. The induction below then propagates them to all sizes. The argument uses the Knuth disjunctive recurrence developed in Smith–Lang and Li–Sun–Zhang [15, 8, 9]. No external increment or monotonicity theorem is needed for the propagation argument.

We use the states, feasibility rules, and three geometric split forms defined in Section 2. Every child is nonempty and strictly smaller than its parent: a simple cut divides $m + n$ elements into two nonempty parts, while a genuine pivot split has two parts of size at least two whose sizes sum to $m + n + 1$. Hence neither pivot child has size more than $m + n - 1$. The six boundary representatives are $00, R0, L0, RL, LL, LR$; reversal supplies the other three states.

3.1 Four auxiliary facts, with proofs

Constraint removal. Removing an endpoint condition can only increase D . Induct on total size. Every split compatible with the stronger condition is compatible with the weaker one. Its corresponding child has either the same conditions or a weaker outer condition, so its value does not decrease by induction. The maximum for each fixed query does not decrease, and neither does the minimum of these query values. Terminal and infeasible states agree with this argument under the conventions above.

Direct sums within the restricted recurrence. Take two nonempty blocks, of sizes (p, q) and (x, y) , and require every first-block element to be smaller than every second-block element. Let λ be a compatible outer left condition and ρ a compatible outer right condition. On the first block retain λ if it is bichromatic, or drop it if it is monochromatic and automatically satisfied; on the second do the same with ρ . Denote the resulting block values by U, V . Then

$$D_{\lambda\rho}(p + x, q + y) \geq U + V. \tag{DS}$$

For two bichromatic blocks this reads $D_{\lambda\rho}(p + x, q + y) \geq D_{\lambda 0}(p, q) + D_{0\rho}(x, y)$.

We prove (DS) by induction on the sum of the two block sizes. A first query with one element in each block has a legal simple response at their boundary, with value $1 + U + V$. If both queried elements lie in the first block, that block is necessarily bichromatic. Use an optimal response in that block. Keep its left child and append the entire second block to its right child. The same split is admissible in the full problem: for a simple split its cut coordinates (p', q') are unchanged, and for a complex split its pivot and cut coordinates are unchanged. The comparison inequalities listed above are therefore unchanged. Its right child has the old inner left condition and the full outer right condition ρ . Apply (DS) inductively to this enlarged right child. This is legitimate even if the old right child is monochromatic: its inherited inner condition is then either absent (simple split) or, for a complex split, both colors are present. The second block carries the compatible outer right condition, exactly as in the statement. The value of the full reply is at least the optimal old reply value plus V , and hence at least $U + V$.

If the first query is inside the second block, prepend the first block to its left child. In that case shift the local cut and, if present, its pivot coordinates by (p, q) . The local query indices shift by the same amounts, so each admissibility inequality is preserved. The inherited outer left condition is compatible with the first block by assumption. Again apply induction to the enlarged child. In either construction the local child has fewer elements than its local parent, so the two-block total decreases. A monochromatic or singleton block has no internal cross-list query; thus it causes no

omitted base case. If the full problem is monochromatic, both block values are zero and the claim is immediate. This proves (DS) purely inside the restricted recurrence; no unrestricted direct-sum theorem is being substituted for it.

Appending a monochromatic tail in (DS) gives, for $u, v \geq 1$,

$$D_{R0}(A, B + v) \geq D_{R0}(A, B + 1), \quad D_{L0}(A + u, B) \geq D_{L0}(A + 1, B). \quad (\text{AX})$$

For zero-length tails the inequalities are equalities. Prepending a monochromatic block similarly shows that $D_{0\rho}(x, y)$ cannot decrease when either dimension is increased, provided the indicated states and outer condition are compatible. Only this explicitly proved one-sided monotonicity will be needed below.

Endpoint deletion. For feasible positive-dimensional states, querying $(1, 1)$ gives

$$D_{R\rho}(m, n) \leq 1 + D_{0\rho}(m, n - 1), \quad D_{L\rho}(m, n) \leq 1 + D_{0\rho}(m - 1, n). \quad (\text{ED})$$

To verify the first bound, the left R condition permits only the answer $a_1 > b_1$. The split inequalities rule out both complex types. A simple response must have $p = 0, q \geq 1$, with a b-only prefix of value zero and suffix value $D_{0\rho}(m, n - q)$. This suffix is bounded above by $D_{0\rho}(m, n - 1)$ by the preceding consequence of (DS). The second bound is symmetric. A zero-dimensional suffix is evaluated by the explicit compatibility convention, so (ED) does not assert an undefined comparison. Reversing both lists gives the corresponding right-endpoint deletion bounds.

A two-element query bound. For $m \geq 3, n \geq 2$ and $\rho \in \{0, L, R\}$, put

$$X_\rho = D_{0\rho}(m - 2, n), \quad Y_\rho = D_{0\rho}(m - 1, n - 1).$$

Whenever the indicated original state is feasible,

$$D_{L\rho}(m, n) \leq \max\{1 + X_\rho, 2 + Y_\rho\}. \quad (\text{Q})$$

This follows by evaluating every response to query $(2, 1)$, not by assuming that the adversary chooses a particular response. For a simple response $a_2 < b_1$, the cut has $p \geq 2, q = 0$. Its prefix is a-only and has value zero. Its suffix, after increasing its a-dimension if necessary, has value at most X_ρ . For a simple response $a_2 > b_1$, compatibility with left L rules out $p = 0$, so $p = 1, q \geq 1$. The prefix has value $D_{L0}(1, q) = 1$, and the suffix has value at most Y_ρ . To see $D_{L0}(1, q) = D_{LL}(1, q) = 1$, query $(1, 1)$: only the simple cut with a-only prefix and b-only suffix is admissible, and one comparison is necessary in a bichromatic nonterminal state.

For a shared a-pivot, the answer $a_2 < b_1$ is impossible since it would require $1 > q \geq 1$. The other answer requires pivot $p = 1$, whose prefix $LR(1, q)$ is infeasible. For a shared b-pivot, the first answer is again impossible. The other forces $p = 1$ and $q \geq 2$. Its prefix is $LL(1, q)$ of value one. After dropping the left R constraint of its suffix, the suffix is bounded by $D_{0\rho}(m - 1, n - q + 1) \leq Y_\rho$. All dimension comparisons here are the one-sided consequences of (DS), with compatible right conditions. These cases exhaust both answers and all three split types, proving (Q).

3.2 The seven boundary checks and the finite translation square

Fix positive integers A, B , put $S = A + B$, and choose

$$H \geq \max\{A + 4, B + 3\}.$$

Require these seven inequalities:

$$\begin{aligned}
D_{00}(A, B) &\geq S - 1, & D_{R0}(A, B) &\geq S - 1, \\
D_{R0}(A, B + 1) &\geq S, & D_{LR}(A + 1, B) &\geq S - 1, \\
D_{L0}(A + 1, B) &\geq S, & D_{L0}(A + 2, B) &\geq S + 1, \\
D_{L0}(A + 1, B + 1) &\geq S + 1.
\end{aligned} \tag{B}$$

In addition, verify throughout $0 \leq m, n \leq H$ that

$$D_{\lambda\rho}(m + A, n + B) \geq D_{\lambda\rho}(m, n) + S \tag{T}$$

for the following domains:

- 00: $m, n \geq 0, m + n \geq 1$;
- R0, L0, RL: positive dimensions and a feasible state;
- LL: positive dimensions, excluding (1, 1);
- LR: positive dimensions and a feasible state, excluding (2, 1).

The excluded whole states are genuine exceptions to the proposed translation and are not silently included in the theorem. The 00 axes have value zero and must be checked if a stored table only lists positive dimensions. The double-zero state is not a translation base.

Theorem 3.1 (Finite translation criterion). *If (B) and the finite square of (T) hold, then (T) holds globally on exactly these domains. This is a simultaneous statement for the six boundary types.*

3.3 Copying a split and locating every exceptional child

Suppose a query in the old (m, n) state has an optimal response with two children. Appending (A, B) to the right child keeps the old cut parameters and any pivot unchanged. All query inequalities remain true. The old right child gains A a-elements and B b-elements. Prepending to the left child instead shifts cut parameters and any pivot from (p, q) to $(p + A, q + B)$; the full query is $(i + A, j + B)$, so the inequalities again remain true. The untouched child has exactly its old dimensions and conditions.

A simple monochromatic child needs special treatment because its original inherited condition was automatically dropped. A b-only prefix with inherited left R enlarges to $D_{R0}(A, B + v) \geq S$ by (AX) and (B); an a-only prefix with left L enlarges to $D_{L0}(A + u, B) \geq S$. The opposite colors would have been incompatible with the original split. Reversal gives the corresponding repairs for a b-only suffix with right L and an a-only suffix with right R . An unconstrained monochromatic child uses the 00 axis translation. All complex children are bichromatic. Therefore no copying step below uses a constrained translation at an undefined axis state.

The following list derives directly from the split rules every exceptional child that can arise in the copy directions used in the six cases below. Here (i, j) are the *old*, unshifted query indices. Other boundary orientations are treated by reversal rather than by additional copy cases.

1. A prefix $RR(1, 1)$ can only be an a-pivot split with $p = q = 1$. The answer $a_i < b_j$ would require $i < 1$ and is impossible. The other answer requires $i \geq 2, j = 1$. The pivot ranges and these query indices force $m, n \geq 2$.
2. A prefix $LL(1, 1)$ can only be a b-pivot split with $p = q = 1$. The first answer gives $i = 1, j \geq 2$; the other would require $j < 1$. Here again the pivot ranges and query indices force $m, n \geq 2$.

3. A prefix $LR(2, 1)$ can only be an a-pivot split with $p = 2, q = 1$. The two answer regions are $i = 1, j \geq 2$ and $i \geq 3, j = 1$. Since $q < n$, this exception requires $n \geq 2$.
4. A suffix $LL(1, 1)$ can only be an a-pivot split with $p = m, q = n - 1$. Its first answer requires $i \leq m - 1, j = n$ and its other answer would require $i > m$. In particular $m, n \geq 2$.

A simple split has an inner 0 condition and thus cannot create any of these doubly constrained exceptions. The list is exhaustive.

We shall repeatedly use this uniform repair for an old state $L\rho$, where $\rho \in \{0, L, R\}$, when the full query satisfies $i \geq A + 3, j = B + 1$ and has arisen from a prefix $LR(2, 1)$. That geometry ensures $m \geq 3, n \geq 2$, so (Q) applies. Set $X = X_\rho, Y = Y_\rho$. If $X \geq 1 + Y$, (DS) at the block sizes $(A + 2, B)$ and $(m - 2, n)$ yields

$$D_{L\rho}(m + A, n + B) \geq D_{L0}(A + 2, B) + X \geq S + 1 + X \geq S + D_{L\rho}(m, n). \quad (\text{Repair-1})$$

This bounds the whole state, so it bounds the adversary value at every first query. It does not presuppose a publicly revealed split or an extra comparison at that boundary. If $X < 1 + Y$, use the simple cut $(A + 1, B + 1)$. It is admissible for this full query because $i > A + 1$ and $j \leq B + 1$. Its value is

$$1 + D_{L0}(A + 1, B + 1) + Y \geq S + 2 + Y \geq S + D_{L\rho}(m, n). \quad (\text{Repair-2})$$

The same repair therefore covers each of the three possible right conditions, including the absence of a right condition.

3.4 Proof of the six global translations

Induct simultaneously on $m + n$, using the finite square as the base. Outside it, $m > H$ or $n > H$. A copied old child is always strictly smaller than the old parent, as proved above. The induction applies to that child, with the explicit monochromatic repairs if appropriate, unless it is one of the exceptional types just listed. Thus neither a nonshrinking recurrence call nor an excluded induction hypothesis is being assumed.

Write $D_{\lambda\rho}^{i,j}$ for the maximum response value at a fixed first query. Showing a lower bound for every such value proves it for D . A lower bound obtained directly for the whole state, as in (Repair-1), also bounds each query value because D is their minimum.

Two elementary coverage observations will be used repeatedly. The four regions relative to the old corner (m, n) are the old rectangle $i \leq m, j \leq n$, the two crossing rectangles, and $i \geq m + 1, j \geq n + 1$. In the last region, $i \leq A, j \leq B$ cannot both hold: they would imply $m \leq A - 1, n \leq B - 1$, inside the finite base. Also, whenever $i \geq A + 1, j \geq B + 1$, the indices $(i - A, j - B)$ belong to the old problem, because $i \leq m + A, j \leq n + B$. Thus all copied old queries below exist. Reversal in the enlarged problem always means the explicit transformation

$$i' = m + A + 1 - i, \quad j' = n + B + 1 - j.$$

Case (a): 00. If $i \leq m, j \geq n + 1$, or $i \geq m + 1, j \leq n$, a simple split into (m, n) and (A, B) has value at least

$$1 + D_{00}(m, n) + D_{00}(A, B) \geq S + D_{00}(m, n).$$

If $i \leq m, j \leq n$, copy an optimal old response and append (A, B) to its right child. Its right constraint is 0, so the nonexceptional 00, L0, R0 translations suffice. If $i \geq m + 1, j \geq n + 1$, the inequalities $i \leq A, j \leq B$ cannot both hold outside the base square. If the reversed query also had $i' \geq m + 1, j' \geq n + 1$, the displayed transformation would give precisely $i \leq A, j \leq B$, a

contradiction. Hence the reversed query lies in one of the first three regions, already treated. State 00 is reversal-invariant. If the old state is on an axis, the old-rectangle query case is empty, and the same crossing or reversal argument still applies; the old block is nonempty because $(m, n) \neq (0, 0)$.

Case (b): $R0$. If $i \leq m, j \geq n + 1$, or $i \geq m + 1, j \leq n$, split into the old block and the added block. The value is at least $1 + D_{R0}(m, n) + D_{00}(A, B) \geq S + D_{R0}(m, n)$. If $i \leq m, j \leq n$, append the added block to the right child of an optimal old response. That child's right condition is 0, so its translation is one of 00, $L0$, $R0$, including the stated axis repairs. No exceptional double-boundary state arises. In the remaining upper-right region, if $i \leq A, j \geq B + 1$ or $i \geq A + 1, j \leq B$, the simple cut gives

$$1 + D_{R0}(A, B) + D_{00}(m, n) \geq S + D_{R0}(m, n).$$

If $i \geq A + 1, j \geq B + 1$, copy the optimal old response to $(i - A, j - B)$ and prepend the block to its left child. The only exception is $RR(1, 1)$, equivalent by reversal to $LL(1, 1)$. Its geometry forces $i \geq A + 2, j = B + 1$. Use instead

$$1 + D_{R0}(A, B + 1) + D_{00}(m, n - 1) \geq S + 1 + D_{00}(m, n - 1) \geq S + D_{R0}(m, n).$$

Case (c): $L0$. For the two crossing regions at the old corner, the simple split has value $1 + D_{L0}(m, n) + D_{00}(A, B) \geq S + D_{L0}(m, n)$. For $i \leq m, j \leq n$, append to the right child of an optimal old response; again only 00, $L0$, $R0$ translations and the axis repairs are needed. In the remaining upper-right region, if $i \leq A, j \geq B + 1$ or $i \geq A + 2, j \leq B$, repeat the pivot a_{A+1} to obtain

$$1 + D_{LR}(A + 1, B) + D_{L0}(m, n) \geq S + D_{L0}(m, n).$$

The missing case $i = A + 1, j \leq B$ forces $m \leq A, n \leq B - 1$ and is in the base. For $i \geq A + 1, j \geq B + 1$, prepend to the left child of an optimal old response. An exceptional child $LL(1, 1)$ forces $i = A + 1, j \geq B + 2$, and is repaired by

$$1 + D_{L0}(A + 1, B) + D_{00}(m - 1, n) \geq S + D_{L0}(m, n).$$

This repair works for every query at that location, including a response whose exceptional child is $LR(2, 1)$.

The only remaining $LR(2, 1)$ location is $i \geq A + 3, j = B + 1$. Apply (Repair-1) or (Repair-2) with $\rho = 0$. This includes $i = A + 3$; there is no omitted equality case at this endpoint.

Case (d): RL . If $i \leq A, j \geq B + 1$ or $i \geq A + 1, j \leq B$, use the simple cut

$$1 + D_{R0}(A, B) + D_{0L}(m, n) \geq S + D_{RL}(m, n).$$

For $i \geq A + 1, j \geq B + 1$, prepend to the old left child. Its only exception is $RR(1, 1)$, forcing $i \geq A + 2, j = B + 1$; replace it by

$$1 + D_{R0}(A, B + 1) + D_{0L}(m, n - 1) \geq S + D_{RL}(m, n).$$

For $i \leq A, j \leq B$, reverse both lists. The state RL is reversal-invariant, and $i' = m + A + 1 - i, j' = n + B + 1 - j$. If also $i' \leq A, j' \leq B$, these equations would imply $m + 1 \leq i \leq A, n + 1 \leq j \leq B$, forcing $m \leq A - 1, n \leq B - 1$, inside the base. Thus the reversed query belongs to one of the other three regions at the (A, B) corner, already covered.

Case (e): LL . The regions $i \leq m, j \geq n + 1$ and $i \geq m + 1, j \leq n$ admit the simple cut

$$1 + D_{L0}(m, n) + D_{0L}(A, B) \geq S + D_{LL}(m, n).$$

For $i \leq m, j \leq n$, append to the right child of an optimal old response. Its only exception is $LL(1, 1)$; the query then has $i \leq m - 1, j = n$. The repair is

$$1 + D_{L0}(m, n - 1) + D_{0L}(A, B + 1) \geq S + D_{LL}(m, n).$$

The last inequality uses right-endpoint deletion and retains the left L constraint, as required by the child state.

In the upper-right case, the repeated-pivot split used in (c) gives

$$1 + D_{LR}(A + 1, B) + D_{LL}(m, n) \geq S + D_{LL}(m, n)$$

when $i \leq A, j > B$ or $i \geq A + 2, j \leq B$. The remaining case $i = A + 1, j \leq B$ is in the finite base. For $i \geq A + 1, j \geq B + 1$, prepend to the old left child. The $LL(1, 1)$ location $i = A + 1, j \geq B + 2$ is repaired by

$$1 + D_{L0}(A + 1, B) + D_{0L}(m - 1, n) \geq S + D_{LL}(m, n).$$

The same repair covers an $LR(2, 1)$ child at that location.

The sole remaining $LR(2, 1)$ location is $i \geq A + 3, j = B + 1$. Use (Repair-1) or (Repair-2) with $\rho = L$. All their hypotheses follow from the exceptional-child geometry already established.

Case (f): LR . This state is reversal-invariant. If $n > H$, choose the orientation with

$$j \geq \left\lceil \frac{B + n + 1}{2} \right\rceil \geq B + 2.$$

For $i \leq A$, repeat a_{A+1} and obtain $1 + D_{LR}(A + 1, B) + D_{LR}(m, n) \geq S + D_{LR}(m, n)$. For $i \geq A + 1$, prepend to the optimal old left child. The $LL(1, 1)$ exception forces $i = A + 1, j \geq B + 2$ and is repaired by

$$1 + D_{L0}(A + 1, B) + D_{0R}(m - 1, n) \geq S + D_{LR}(m, n).$$

That same cut covers any $LR(2, 1)$ exception at this location. Its only remaining location has $j = B + 1$, excluded by the orientation.

If $n \leq H$, then $m > H$. Choose the reversal with

$$i \geq \left\lceil \frac{A + m + 1}{2} \right\rceil \geq A + 2.$$

For $j \leq B$, repeat a_{A+1} as above. For $j \geq B + 1$, prepend to the old left child. The $LL(1, 1)$ exception would require $i = A + 1$ and is impossible. For the $LR(2, 1)$ exception, the location $i = A + 1$ is excluded by orientation, so the only possibility is $i \geq A + 3, j = B + 1$. Use (Repair-1) or (Repair-2) with $\rho = R$.

This completes the simultaneous induction. Every copied nonexceptional child is strictly smaller, every monochromatic child was handled by an axis repair, and every occurrence of either exceptional state was explicitly repaired. The exceptional whole states are never invoked by induction.

3.5 A finite cone of residual sizes

The last step must not assume that tape optimality is monotone in one list length. That unrestricted monotonicity is related to a classical open conjecture. Instead we use the translation in both coordinate directions. The next elementary lemma reduces every pair in a cone to a bounded pair in the same cone.

Lemma 3.2 (Finite cone remainder). *Let $1 \leq A < B$ and $S = A + B$. If positive integers m, n satisfy*

$$Am \leq Bn, \quad An \leq Bm,$$

then there are nonnegative integers k, ℓ and positive integers $r, t < S$ such that

$$(m, n) = k(A, B) + \ell(B, A) + (r, t), \quad Ar \leq Bt, \quad At \leq Br.$$

Proof. Put $\Delta = B^2 - A^2 > 0$ and define

$$x = \frac{Bn - Am}{\Delta}, \quad y = \frac{Bm - An}{\Delta}.$$

The cone inequalities imply $x, y \geq 0$, and direct multiplication gives $(m, n) = x(A, B) + y(B, A)$. Initially take $k = \lfloor x \rfloor$, $\ell = \lfloor y \rfloor$, and set

$$(r, t) = (m, n) - k(A, B) - \ell(B, A) = \{x\}(A, B) + \{y\}(B, A),$$

where $\{z\} = z - \lfloor z \rfloor$ is the fractional part. Thus r, t are integers in $[0, S)$ and are nonnegative combinations of the two bounding vectors. They satisfy the same cone inequalities. If either fractional part is nonzero, positivity of all four vector coordinates implies $r, t > 0$. If both fractional parts vanish, then $(r, t) = (0, 0)$ and $k + \ell > 0$, since $m, n > 0$. If $k > 0$, replace k by $k - 1$ and (r, t) by (A, B) ; otherwise replace ℓ by $\ell - 1$ and (r, t) by (B, A) . The new residual is positive, has both coordinates below S , and remains in the cone. This last adjustment is essential: the translation theorem was not asserted at the empty state $(0, 0)$. $\square$

Theorem 3.3 (Tape optimality from finite translation and cone checks). *Suppose $1 \leq A < B$, the hypotheses of Theorem 3.1 hold, and the following finite equalities have been verified:*

$$D_{00}(r, t) = r + t - 1 \quad \text{for } 1 \leq r \leq t < S, \quad At \leq Br. \quad (\text{C})$$

Then

$$M(m, n) = m + n - 1 \quad \text{whenever } m \leq n \leq \frac{B}{A}m.$$

Proof. The unconstrained translation in Theorem 3.1 gives

$$D_{00}(u + A, v + B) \geq D_{00}(u, v) + S$$

for every $u, v \geq 0$ with $u + v \geq 1$. Symmetry under swapping the two lists also gives

$$D_{00}(u + B, v + A) \geq D_{00}(u, v) + S.$$

For the desired (m, n) , both inequalities defining the cone hold. Use Lemma 3.2. By (C) and list-swap symmetry, $D_{00}(r, t) = r + t - 1$ also when $r > t$. Starting with this positive residual, iterate the first translation k times and the second ℓ times. All intermediate dimensions are positive. It follows that

$$D_{00}(m, n) \geq (k + \ell)S + r + t - 1 = m + n - 1.$$

The adversary lower bound $D_{00} \leq M$, established by the game-semantics argument, and the ordinary tape-merge upper bound $M \leq m + n - 1$ force equality. No diagonal-extension, constrained-increment, or deficit-monotonicity lemma is used. $\square$

3.6 The concrete parameters and the finite obligations

For $(A, B, H) = (95, 148, 152)$, we have $S = 243$ and $H \geq \max\{A + 4, B + 3\}$. The finite square uses indices no larger than $(247, 300)$ and so is contained in a table computed through dimensions 300. The seven boundary thresholds in (B), in their displayed order, are

$$242, \quad 242, \quad 243, \quad 242, \quad 243, \quad 244, \quad 244.$$

The cone check (C) consists of 10,651 pairs. Equivalently, checking both orders uses 21,060 pairs, since the 242 diagonal pairs are counted once rather than twice. The finite translation square has 138,622 feasible, nonexcluded inequalities. These exact finite obligations, together with the recurrence definition and the induction just proved, imply

$$M(m, n) = m + n - 1 \quad \text{for} \quad m \leq n \leq \frac{148}{95}m.$$

The separate computation section records how each obligation is evaluated and independently checked.

4 The finite computation and reproduction

4.1 Evaluating the recurrence exactly

We evaluated all nine states with $1 \leq m, n \leq 300$, in increasing order of $m + n$. Every child in a legal response has smaller total size, so each lookup refers to a value already proved by the recurrence. There are $9 \cdot 300^2 = 810\,000$ stored entries, including 600 impossible states. All arithmetic is integral. Impossible states are represented in the files by -10000 ; a response with such a child is discarded before addition. Every feasible value lies between 1 and $m + n - 1 \leq 599$ by (3), so the signed 16-bit storage of the primary program is sufficient. Zero-axis values are supplied by the explicit normalization rules of Section 2.

The primary program, `adversary/disjunctive_fast.cpp`, uses the three geometric split forms. Each split supplies its child sum to one or two rectangles of possible first queries. For example, a simple cut (p, q) applies to $i \leq p, j \geq q + 1$ and $i \geq p + 1, j \leq q$. Store the sum at the corner of each applicable rectangle, then propagate maxima in the corresponding two coordinate directions. The resulting entry at (i, j) is exactly the greatest response sum available there. Taking one plus the least of these entries implements (5). Enumerating split corners and propagating rectangle maxima both cost $O(mn)$ operations per boundary state, rather than enumerating every split separately for every query. Summing over dimensions at most N gives $O(N^4)$ operations and $O(N^2)$ stored values; the factors for the nine boundary types are constant.

A separate implementation, `dp_audit/fast_literal.cpp`, was written from the six published formulas without consulting the primary source. It computes all nine boundary types without symmetry reduction. It keeps six rectangle-max arrays, one per formula. In the order of the rows in Section 2, their query entries are $(i, j), (i, j - 1), (i + 1, j), (i, j), (i, j + 1), (i - 1, j)$. The strict offsets enforce the pivot exclusions. Cells that would refer to the current parent but cannot be queried are omitted. The independently generated complete tables agree byte for byte. A slower literal implementation, `dp_audit/literal.cpp`, enumerates all six parameter ranges directly. Its 15,912 entries on $1 \leq m \leq 34, 1 \leq n \leq 52$ also agree with the primary table.

The common complete CSV file has the SHA-256 digest

254a1a1c9efc1b7efe96585934ba7138
57840f4230bc9388874017845739307a

The two displayed lines concatenate to one digest. A digest identifies the data; it is not a substitute for computing or checking the recurrence. The CSV columns are `m,n,l,r,value`, with $0, L, R$ encoded as $0, 1, 2$.

4.2 The checked hypotheses

The final checker, `audit_arxiv/verify_main_theorem.py`, first validates the table’s dimensions, distinct row keys, feasibility sentinels, and integer ranges. It then checks exactly (B), (T), and (C), with $(A, B, H) = (95, 148, 152)$. The seven boundary values, all attaining their required thresholds, are

State and dimensions	Computed value	Required minimum
$D_{00}(95, 148)$	242	242
$D_{R0}(95, 148)$	242	242
$D_{R0}(95, 149)$	243	243
$D_{LR}(96, 148)$	242	242
$D_{L0}(96, 148)$	243	243
$D_{L0}(97, 148)$	244	244
$D_{L0}(96, 149)$	244	244

For each translation define its slack to be the left side of (T) minus the right side. The counts below include only the stated feasible, nonexcluded domains.

Boundary type	Inequalities checked	Minimum slack
00	23,408	0
L0	23,104	0
R0	23,104	0
RL	22,952	0
LL	23,103	0
LR	22,951	0
Total	138,622	0

The excluded translations from $LL(1, 1)$ and $LR(2, 1)$ each have slack -1 . Thus omitting their repairs would invalidate the proof. Finally the checker tests all 10,651 pairs $1 \leq r \leq t \leq 242$ satisfying $95t \leq 148r$ and finds $D_{00}(r, t) = r + t - 1$ in every case. List-swap symmetry covers the reverse ordering of the residual coordinates. Theorem 3.3 now proves Theorem 1.1.

4.3 What is and is not certified

The finite program proves integer statements about the recurrence. Section 2 proves that the recurrence supplies a lower bound for ordinary merging; Section 3 proves the passage from finite checks to all dimensions. None of these three steps can replace either of the others. In particular a value $D_{00}(m, n) < m + n - 1$ does not prove that tape merge is suboptimal: the restricted adversary may simply be too weak.

The accompanying code is a reproducible computational certificate, not a proof formalized in a proof assistant. The independent implementations, literal cross-checks, and separate certificate verifier reduce the risk of an implementation error. Readers can rerun the finite checks or regenerate the tables from the displayed recurrences. No floating-point estimates or numerically inferred asymptotic regularities enter the proof.

5 An unrestricted finite certificate for exact merging

We work with deterministic comparison decision trees and distinct keys. The input consists of two internally ordered chains $a_1 < \dots < a_m$ and $b_1 < \dots < b_n$. A partial order P records the two chain orders and all relations implied by the comparisons already answered. A linear extension of P is a total order consistent with all these relations. Two keys are incomparable if neither order between them is implied. At an internal node the algorithm may compare any two keys; comparisons whose outcomes are already implied can be deleted. Thus it suffices to consider incomparable pairs, necessarily with one key in each chain. The cost is the maximum root-to-leaf number of comparisons. Let $H(P)$ denote the optimum remaining cost for a partial order P obtained from the two chains and an arbitrary consistent comparison history. In particular, $M(m, n) = H(P_{m,n})$ for the disjoint union of the two chains. All lower-bound arguments below concern this unrestricted model.

5.1 Exact states and their legal queries

For a total order extending the two chains, let x_i be the number of b-keys preceding a_i . The total order is uniquely specified by

$$0 \leq x_1 \leq \dots \leq x_m \leq n.$$

A state is a pair of nondecreasing integer vectors $\ell = (\ell_1, \dots, \ell_m)$ and $u = (u_1, \dots, u_m)$ satisfying

$$0 \leq \ell_i \leq u_i \leq n \quad (1 \leq i \leq m). \quad (6)$$

It represents exactly the rank vectors with $\ell_i \leq x_i \leq u_i$. Equivalently, it contains the cross-chain relations

$$b_j < a_i \quad (j \leq \ell_i), \quad a_i < b_j \quad (j > u_i).$$

The original merging problem has $\ell_i = 0$ and $u_i = n$. For example, with $m = n = 2$, the rank vector $(0, 2)$ describes the order $a_1 < b_1 < b_2 < a_2$. The answer $a_1 < b_2$ restricts x_1 to at most 1, while $b_2 < a_1$ restricts it to at least 2.

Lemma 5.1 (State sufficiency and query completeness). *Every consistent history of comparisons between the two chains has the above representation. In a represented state, a_i and b_j are incomparable if and only if $\ell_i < j \leq u_i$. Both outcomes of each such comparison are feasible. Its two successor states are*

$$u_k^- = \begin{cases} \min(u_k, j-1), & k \leq i, \\ u_k, & k > i, \end{cases} \quad \ell^- = \ell, \quad (7)$$

$$\ell_k^+ = \begin{cases} \ell_k, & k < i, \\ \max(\ell_k, j), & k \geq i, \end{cases} \quad u^+ = u. \quad (8)$$

The minus successor means $a_i < b_j$ and the plus successor means $b_j < a_i$.

Proof. The outcomes of a comparison impose $x_i \leq j-1$ or $x_i \geq j$. Monotonicity propagates these inequalities precisely as in (7)–(8); it introduces no disjunctive restriction. Induction from the unrestricted initial state proves sufficiency. Conversely, each feasible rank vector constructs one consistent total order, by putting a_i in the gap after b_{x_i} , retaining the order of the a-keys when several ranks agree. The vectors ℓ and u themselves are feasible. For an incomparable query, $\ell_i < j$ and $u_i \geq j$ show that these two vectors witness the two possible outcomes. The updated endpoints remain nondecreasing and satisfy (6); for example, when $k \leq i$, $\ell_k \leq \ell_i \leq j-1$ in the minus case. All comparisons outside the stated range have fixed outcomes, as do all within-chain comparisons, so the listed queries are exhaustive. $\square$

Write

$$\eta(P) = \sum_{i=1}^m (u_i - \ell_i). \quad (9)$$

This is exactly the number of incomparable pairs. Each legal query strictly decreases η in both successors: at least the queried pair becomes comparable, and no known relation is removed.

Swapping the chains and reversing the whole total order preserve H . For reversal the endpoints are $\ell'_i = n - u_{m+1-i}$ and $u'_i = n - \ell_{m+1-i}$. After swapping the chains, the interval for b_j , now in the first chain, has endpoints

$$\ell'_j = \#\{i : u_i < j\}, \quad u'_j = \#\{i : \ell_i < j\}. \quad (10)$$

Here $\#$ means the number of indices satisfying the stated condition. These operations commute and generate four equivalent representations. The implementation takes their lexicographically least encoding (the first in dictionary order); this is only a symmetry reduction and imposes no restriction on the query set.

5.2 Ordinal-sum decomposition

Let $G(P)$ be the graph whose vertices are the keys and whose edges are the incomparable pairs. A connected component is a maximal set of vertices joined by paths of edges; it is nontrivial if it contains at least two vertices. The neighbors of a_i are exactly $b_{\ell_i+1}, \dots, b_{u_i}$. The nontrivial connected components can therefore be found by grouping successive nonempty row intervals whenever they overlap. A group beginning at row r and ending at row s has second-chain indices $\ell_r + 1, \dots, u_s$; its normalized endpoints are $\ell_i - \ell_r, u_i - \ell_r$, for $r \leq i \leq s$. A new group begins if the next row is empty or its left endpoint is at least the preceding right endpoint. Monotonicity of both endpoint sequences ensures that no later row can bridge such a separation. Isolated vertices are retained in the determined output order and contribute zero cost.

Lemma 5.2 (Component additivity). *If $P_1, \dots, P_t$ are the nontrivial components just described, then*

$$H(P) = \sum_{r=1}^t H(P_r). \quad (11)$$

Proof. First, connected components of an incomparability graph are ordinal-sum factors. Indeed, vertices from different components are comparable. For a fixed vertex y outside a component, its comparison direction with vertices of that component cannot change along an incomparability edge: $x < y < z$ would imply $x < z$. Propagating along a path proves that y lies entirely before or entirely after the component. Applying this to both components shows that their direction is uniform. These directions give a linear order of the components. Their internal linear extensions can consequently be chosen independently.

This ordered concatenation of independently ordered parts is called an ordinal sum. An upper bound is obtained by solving the components separately. For the reverse inequality, queries between different components have known answers and can be discarded. In each component, the adversary can maintain its local optimal remaining height. For any query in a component of height h , some answer leaves local height at least $h - 1$; otherwise that query would give an algorithm of height less than h . The adversary uses this answer in the queried component and leaves all others unchanged. The sum of the local heights decreases by at most one per comparison and must be zero when the whole order is determined. Initially it is the right side of (11). This proves the lower bound even for algorithms that interleave queries among components. $\square$

There is no nontrivial component exactly when P is a total order, in which case $H(P) = 0$. The sum of the areas of all components is $\eta(P)$. In particular, every component of either successor of a legal query has area strictly smaller than that of the original state.

5.3 Elementary bounds with complete recurrences

Let $e(P)$ be the number of linear extensions. The indicator $\mathbf{1}_E$ is 1 when the condition E holds and 0 otherwise. It is computed without enumerating them by setting $d_0(0) = 1$, $d_0(j) = 0$ for $j > 0$, and

$$d_i(j) = \mathbf{1}_{\{\ell_i \leq j \leq u_i\}} \sum_{r=0}^j d_{i-1}(r), \quad e(P) = \sum_{j=0}^n d_m(j). \quad (12)$$

Here $d_i(j)$ counts feasible rank prefixes ending at $x_i = j$; the recurrence follows by conditioning on x_{i-1} . Thus the decision-tree leaf bound gives

$$H(P) \geq I(P) := \lceil \log_2 e(P) \rceil. \quad (13)$$

For a linear extension π of P , let $w_P(\pi)$ be the number of adjacent pairs in π that are incomparable in P . Such a pair necessarily consists of keys from different chains. Set $W(P) = \max_{\pi} w_P(\pi)$.

Lemma 5.3 (Unresolved adjacency bound). *For every represented state P , one has $H(P) \geq W(P)$.*

Proof. Fix an extension π . If adjacent keys x, y in π are incomparable in P , swapping them gives another extension of P . Every comparison other than the comparison of x and y has the same outcome on these two inputs. A correct algorithm must therefore compare x with y on its execution on π . This applies separately to every counted adjacent pair, and proves that this execution has length at least $w_P(\pi)$. Maximizing over π proves the lemma. Pairs already comparable in P are not counted. $\square$

Here are explicit polynomial-time recurrences for W and for an elementary upper bound. A prefix of a merged order is described by the numbers (i, j) of emitted a- and b-keys. Put $\ell_0 = 0$ and $u_{m+1} = n$. A feasible prefix vertex satisfies $\ell_i \leq j \leq u_{i+1}$. From a feasible vertex, the permissible next moves are

$$\begin{aligned} \mathcal{A}(i, j) : \quad & i < m, \quad \ell_{i+1} \leq j \leq u_{i+1}, \\ \mathcal{B}(i, j) : \quad & j < n, \quad \ell_i \leq j + 1 \leq u_{i+1}. \end{aligned}$$

Every root-to-terminal path using these moves corresponds to one extension of P , and every extension gives such a path. This follows either by the rank-vector bijection or by checking the required rank of each newly emitted a-key. Only vertices reachable from $(0, 0)$ are needed.

Let $F_A(i, j)$ and $F_B(i, j)$ be the maximum future contribution to $w_P(\pi)$, given a prefix ending in an a-key or a b-key, respectively. The switch weights are

$$\alpha(i, j) = \mathbf{1}_{\{j > 0, \ell_{i+1} < j \leq u_{i+1}\}}, \quad \beta(i, j) = \mathbf{1}_{\{i > 0, \ell_i < j + 1 \leq u_i\}}.$$

The first is used only for a permissible $\mathcal{A}$ move, and the second only for a permissible $\mathcal{B}$ move. With both terminal values zero, backward dynamic programming gives

$$F_A(i, j) = \max(\{F_A(i+1, j) : \mathcal{A}(i, j)\}, \{\beta(i, j) + F_B(i, j+1) : \mathcal{B}(i, j)\}), \quad (14)$$

$$F_B(i, j) = \max(\{\alpha(i, j) + F_A(i+1, j) : \mathcal{A}(i, j)\}, \{F_B(i, j+1) : \mathcal{B}(i, j)\}). \quad (15)$$

An unavailable option is omitted from the maximum. At the empty prefix the switch weights for the first move are zero, and hence

$$W(P) = \max\{F_A(0, 0), F_B(0, 0)\}.$$

These formulas explicitly count only unresolved cross-chain adjacencies.

For an upper bound, use ordinary front-element merging, omitting a comparison when the next key is already forced. If exactly one of the two moves is permissible, emit that next key for free. If both are permissible, compare the two front keys and emit the smaller one. Let $T(m, n) = 0$, and set

$$T(i, j) = \mathbf{1}_{\{\mathcal{A}(i, j), \mathcal{B}(i, j)\}} + \max(\{T(i+1, j) : \mathcal{A}(i, j)\}, \{T(i, j+1) : \mathcal{B}(i, j)\}) \quad (16)$$

for a nonterminal feasible prefix. Then $H(P) \leq T(0, 0)$. The comparison just used involves the emitted key, so it introduces no additional restriction among the un-emitted keys beyond the original state and the enlarged prefix. Consequently (16) is the exact worst-case cost of this specified algorithm.

For a nontrivial indecomposable state with one chain of size one, every key of the other chain is incomparable with it. There are exactly $n+1$ possible insertion gaps. Binary insertion and the leaf bound give the exact value $\lceil \log_2(n+1) \rceil$, with the analogous formula after exchanging the chains. We use this exact value in place of the front-merge upper bound. Thus the elementary bounds $E_-(P), E_+(P)$ used below are

$$E_-(P) = \max\{I(P), W(P)\}, \quad E_+(P) = T(0, 0),$$

with this one-element-chain refinement. For a decomposable state, we apply the bounds to its nontrivial components and sum them.

5.4 A finite, well-founded certificate

For a nonterminal state, the unrestricted decision-tree recurrence is

$$H(P) = 1 + \min_{\ell_i < j \leq u_i} \max\{H(P_{ij}^-), H(P_{ij}^+)\}. \quad (17)$$

The subscripts ij denote the successors of query $a_i : b_j$. The root of any decision tree makes one of the listed queries, and its two subtrees solve the two successors. Conversely, combining a query with optimal successor trees constructs a valid tree. This proves (17), including both inequalities.

A certificate assigns integer bounds $L(P), U(P)$ to finitely many canonical, nontrivial indecomposable states. All other such states receive the explicit defaults $E_-(P), E_+(P)$. For any state Q , define

$$\mathcal{L}(Q) = \sum_{C \in \text{comp}(Q)} L(C), \quad \mathcal{U}(Q) = \sum_{C \in \text{comp}(Q)} U(C),$$

where $\text{comp}(Q)$ denotes the set of nontrivial normalized components, and a missing component record is interpreted using the defaults. The empty sum is zero. Each explicit record must be a valid canonical state and satisfy $0 \leq L(P) \leq U(P)$. Its two assertions are checked as follows:

1. If $L(P) > E_-(P)$, check *every* legal query and require

$$1 + \max\{\mathcal{L}(P_{ij}^-), \mathcal{L}(P_{ij}^+)\} \geq L(P). \quad (18)$$

2. If $U(P) < E_+(P)$, check that *at least one* legal query satisfies

$$1 + \max\{\mathcal{U}(P_{ij}^-), \mathcal{U}(P_{ij}^+)\} \leq U(P). \quad (19)$$

When a proposed bound is weaker than its corresponding elementary bound, no Bellman check is needed for that bound.

Theorem 5.4 (Certificate soundness). *If these checks pass, every explicit and default assertion is true: $L(P) \leq H(P) \leq U(P)$.*

Proof. Induct on the nonnegative integer $\eta(P)$ in (9). Defaults are true by the elementary bounds, regardless of the area. For an explicit assertion not established by a default bound, every component of every successor appearing in (18) or (19) has strictly smaller area. By the induction hypothesis and Lemma 5.2 its aggregated bounds are true. Equation (17) now proves the requested lower or upper bound. The base case is a total order, with area and cost zero. This also shows that it is unnecessary to list or verify the certificate records in topological order: their logical dependency is nevertheless acyclic. Missing records cannot create an unsupported assertion because their defaults have independent proofs. $\square$

The upper certificate is constructive. At an explicit state with an improved upper bound, enumerate its queries and use a witness to (19). After the observed outcome, solve its ordinal-sum components recursively. At a default upper bound use the front algorithm (16), or binary insertion in the one-element case. Canonicalization only requires tracking the chain exchange and order reversal when translating indices and outcomes back to the current keys. Strict area decrease proves termination, and the checked inequalities prove the asserted worst-case cost. The lower certificate similarly records, implicitly or explicitly, a hard outcome for every legal query. Its elementary leaves are justified by Lemma 5.3 or the decision-tree leaf count, not by a restriction on allowed algorithms.

5.5 The two checked instances

Applying this procedure to the supplied integer certificates yields

$$M(6, 22) = 21, \quad M(6, 25) = 22. \quad (20)$$

The finite verification statistics are

(m, n)	explicit records	default states evaluated	certified root interval
(6, 22)	454 963	5 560 670	[21, 21]
(6, 25)	1 111 896	13 522 983	[22, 22].

The verifier is a standalone program: it neither imports nor runs the search program. It evaluates (12), the backward recurrences (14)–(16), and all the required Bellman inequalities. The readable verifier validates endpoint inequalities, monotonicity, dimensions, indecomposability, integer ranges, canonical form and uniqueness of every explicit record before checking its assertions. Its requirements use exceptions and cannot be disabled by compiler assertion flags. Both full passes were also run with assertions disabled. A separate streaming check independently validates the endpoint and range contracts for both files. All encountered states have at most 31 keys and at most six keys in the shorter chain. Their extension counts are at most $\binom{31}{6} = 736\,281$, so the exact unsigned 64-bit arithmetic used for counting cannot overflow in these certificates. The largest asserted height is 30, and all endpoint and height encodings fit their specified byte fields. No published numerical merging values are used as default bounds or seeds.

As an additional software check, an independently written solver on explicit sets of interleavings, using neither component decomposition nor symmetry reduction nor the adjacency bound, agrees on all 940 consistent interval states with $1 \leq m \leq 3$ and $1 \leq n \leq 4$. This finite cross-check supplements, and does not replace, the proofs of the reductions and the complete certificate verification.

For $(6, 22)$, a certified optimal first query is a_3 versus b_{10} . The two constrained successors both have upper bound 20; their reflected counterpart gives the query a_4 versus b_{13} . The lower certificate checks all $6 \cdot 22 = 132$ possible first queries and in each case certifies an outcome requiring at least 20 more comparisons. These are certified optimal first queries, without a claim that they exhaust all optimal choices.

6 Further directions

The main result uses a larger block within the existing disjunctive framework. It therefore leaves two different questions open: how far block translations can be extended within that framework, and how much stronger a different adversary can be. The obstruction proved by Li–Sun–Zhang near ratio 1.8 concerns this particular lower-bound method, not an algorithm proving tape merge suboptimal at that ratio.

Larger-looking numerical blocks must be checked against the complete translation hypothesis, including small constrained states. For example, the candidate blocks $(91, 142)$ and $(93, 145)$ both fail the proposed translation at $RL(1, 2)$, $LL(2, 2)$, and $LR(3, 2)$, each with slack -1 . Thus an improved ratio cannot be inferred from a few favorable block boundary values. Additional exceptional-state repairs, a different translation scheme, or a stronger game would be needed.

The exact certificates illustrate a complementary direction. Their lower bounds quantify over every legal comparison, while their upper bounds describe actual algorithms. They settle the two finite equalities in Theorem 1.2, but do not establish a formula for all $M(6, n)$. In particular, Smith–Lang’s further implications involving $(8, 29)$ also need a separate assertion about $(7, 22)$; the two equalities proved here alone do not resolve that case. Neither the exact computations nor the upper-bound appendix solve the global merging problem.

A An upper-bound consequence of the modified binary merge

This appendix records a consequence of the finite algorithms of Smith–Lang [15] and the modified binary merge of Li–Sun–Zhang [8]. Smith–Lang, Section 5.1, already conjectured the equalities

$$M(m, 2m - 3 + k) = 3m - 5 + k \quad (m \geq 9, 0 \leq k \leq 2).$$

We prove their upper-bound direction. We make no claim here about the matching lower bounds or the priority of this consequence. The recursive argument is included in full; the finite comparison algorithms explicitly listed below are used as established external results.

We put $M(m, 0) = M(0, n) = 0$. All list elements are distinct, and all comparisons are comparisons between elements of the two initially sorted lists.

Lemma A.1 (Elementary bounds). *For $m, n \geq 1$,*

$$M(m, n) \leq m + n - 1.$$

For $m \geq 1$ and $n \geq 2$,

$$M(m, n) \leq 2m + \lfloor n/2 \rfloor - 1. \tag{21}$$

Proof. The first assertion is tape merge. For the second, first merge the m a-elements with the $\lfloor n/2 \rfloor$ even-indexed b-elements by tape merge. This uses at most $m + \lfloor n/2 \rfloor - 1$ comparisons. Each gap between consecutive even-indexed elements, including the two end gaps, contains at most one omitted odd-indexed b-element. If that gap contains t a-elements, insert its omitted element using at most t comparisons. Summing over the gaps costs at most m more comparisons. $\square$

Lemma A.2 (Li–Sun–Zhang recurrence). *For $m, n \geq 2$,*

$$M(m, n) \leq \max\{M(m, n-2) + 1, M(m-1, n-2) + 3, \\ M(m-2, n) + 3, M(m-2, n-1) + 4\}. \quad (22)$$

Proof. Compare a_1 with b_2 . If $b_2 < a_1$, output b_1, b_2 and continue with sizes $(m, n-2)$, having spent one comparison. Otherwise compare a_2 with b_2 . If $b_2 < a_2$, compare a_1 with b_1 ; the three-element prefix consisting of a_1, b_1, b_2 is then ordered, leaving sizes $(m-1, n-2)$ after three comparisons. In the remaining case $a_2 < b_2$, compare a_2 with b_1 . If $a_2 < b_1$, output a_1, a_2 , leaving $(m-2, n)$ after three comparisons. If $b_1 < a_2$, compare a_1 with b_1 and output the ordered prefix consisting of a_1, a_2, b_1 , leaving $(m-2, n-1)$ after four comparisons. Any additional information about the remaining lists can be discarded. $\square$

Finite inputs. The following twelve finite upper bounds are recorded in the table of Smith–Lang [15]; several had been established earlier. Only upper bounds are needed.

$$\begin{array}{c|cccc} (m, n) & (3, 6) & (3, 8) & (3, 10) & (3, 12) \\ \hline \text{upper bound} & 7 & 8 & 9 & 10 \\ \\ (m, n) & (5, 9) & (5, 11) & (5, 13) & \\ \hline \text{upper bound} & 12 & 13 & 14 & \\ \\ (m, n) & (7, 12) & (10, 20) & (9, 15) & (10, 17) & (11, 19) \\ \hline \text{upper bound} & 17 & 27 & 22 & 25 & 28. \end{array} \quad (23)$$

We do not regard the published abbreviated drawings, whose leaves can refer to omitted subtrees, as complete independently verified decision-tree certificates. Thus the finite assertions in (23) are the precise external input to this appendix.

Lemma A.3 (The finite strips needed below). *The bounds*

$$M(m, 2m + 2k) \leq 3m + k - 2 \quad (m \geq 3, \quad k \in \{0, 1, 2, 3\}), \quad (24)$$

$$M(m, 2m + 2k - 1) \leq 3m + k - 3 \quad (m \geq 5, \quad k \in \{0, 1, 2\}) \quad (25)$$

follow from the finite inputs and Lemmas A.1 and A.2.

Proof. For (24), use the four $m = 3$ inputs and induction on $m + k$ for $m \geq 4$. Apply (22) at $n = 2m + 2k$. Its first term is at most $3m + k - 2$ by the induction hypothesis with $k - 1$ when $k \geq 1$. When $k = 0$, tape merge gives that same bound: $M(m, 2m - 2) + 1 \leq 3m - 2$. The second term is at most $3(m - 1) + k - 2 + 3 = 3m + k - 2$ by induction on m . For the last two terms, put $r = m - 2 \geq 2$ and use (21):

$$M(r, 2r + 4 + 2k) + 3 \leq 2r + (r + 2 + k) - 1 + 3 = 3m + k - 2,$$

$$M(r, 2r + 3 + 2k) + 4 \leq 2r + (r + 1 + k) - 1 + 4 = 3m + k - 2.$$

This proves the even strip in its stated finite range of k .

For (25), use the three $m = 5$ inputs and again induct on $m + k$, now for $m \geq 6$. In the first term of the recurrence use the odd-strip induction hypothesis with $k - 1$ if $k \geq 1$; at $k = 0$ use $M(m, 2m - 3) + 1 \leq 3m - 3$ from tape merge. The second term uses the odd-strip hypothesis at

$m - 1$. Both are at most $3m + k - 3$. Set $r = m - 2 \geq 4$ in the remaining terms. The elementary pair bound gives

$$M(r, 2r + 3 + 2k) + 3 \leq 3r + k + 3 = 3m + k - 3.$$

The already proved even strip, with index $k + 1 \in \{1, 2, 3\}$, gives

$$M(r, 2r + 2 + 2k) + 4 \leq 3r + (k + 1) - 2 + 4 = 3m + k - 3.$$

All dimensions are positive and all applications lie in the displayed parameter ranges. $\square$

Lemma A.4 (Two further diagonals). *One has*

$$M(m, 2m - 2) \leq 3m - 4 \quad (m \geq 7), \quad (26)$$

$$M(m, 2m) \leq 3m - 3 \quad (m \geq 10). \quad (27)$$

Proof. The first inequality starts from $M(7, 12) \leq 17$. For $m \geq 8$, apply the recurrence with $n = 2m - 2$. Tape merge, induction, the even strip with index 1, and the odd strip with index 1, respectively, bound its four terms by

$$\begin{aligned} M(m, 2m - 4) + 1 &\leq 3m - 4, \\ M(m - 1, 2m - 4) + 3 &\leq 3(m - 1) - 4 + 3 = 3m - 4, \\ M(m - 2, 2m - 2) + 3 &\leq 3(m - 2) + 1 - 2 + 3 = 3m - 4, \\ M(m - 2, 2m - 3) + 4 &\leq 3(m - 2) + 1 - 3 + 4 = 3m - 4. \end{aligned}$$

The strip applications have $m - 2 \geq 6$, so their size conditions hold.

The second inequality starts from $M(10, 20) \leq 27$. For $m \geq 11$, apply the recurrence with $n = 2m$. Use (26), induction, the even strip with index 2, and the odd strip with index 2:

$$\begin{aligned} M(m, 2m - 2) + 1 &\leq 3m - 3, \\ M(m - 1, 2m - 2) + 3 &\leq 3(m - 1) - 3 + 3 = 3m - 3, \\ M(m - 2, 2m) + 3 &\leq 3(m - 2) + 2 - 2 + 3 = 3m - 3, \\ M(m - 2, 2m - 1) + 4 &\leq 3(m - 2) + 2 - 3 + 4 = 3m - 3. \end{aligned}$$

Here $m - 2 \geq 9$, again within the strip domains. $\square$

Define

$$\alpha(m) = \max\{n \geq m : M(m, n) = m + n - 1\}.$$

The set is nonempty since tape merge is optimal at $n = m$, and finite because (21) is strictly below tape cost for $n \geq 2m + 1$.

Proposition A.5 (Upper direction of the Smith–Lang pattern). *For every integer $m \geq 9$ and $k \in \{0, 1, 2\}$,*

$$M(m, 2m - 3 + k) \leq 3m - 5 + k. \quad (28)$$

In particular, tape merge is not optimal whenever $m \geq 9$ and $n \geq 2m - 3$. Consequently $\alpha(m) \leq 2m - 4$ for $m \geq 9$.

Proof. For $k = 0$, the cases $m = 9, 10, 11$ are the final three finite inputs. Suppose $m \geq 12$ and use the recurrence at $n = 2m - 3$. Tape merge, induction on m , the odd strip with index 1, and (27), respectively, give

$$\begin{aligned} M(m, 2m - 5) + 1 &\leq 3m - 5, \\ M(m - 1, 2m - 5) + 3 &\leq 3(m - 1) - 5 + 3 = 3m - 5, \\ M(m - 2, 2m - 3) + 3 &\leq 3(m - 2) + 1 - 3 + 3 = 3m - 5, \\ M(m - 2, 2m - 4) + 4 &\leq 3(m - 2) - 3 + 4 = 3m - 5. \end{aligned}$$

The last application is valid because $m - 2 \geq 10$. This proves the $k = 0$ case. The case $k = 1$ is (26), and $k = 2$ is the odd strip with index 0.

Each of the three bounds saves one comparison over tape merge. At $n = 2m$, the even strip with index 0 gives $M(m, 2m) \leq 3m - 2$, saving one comparison for every $m \geq 9$. For every $n \geq 2m + 1$, the elementary pair bound gives

$$M(m, n) \leq 2m + \lfloor n/2 \rfloor - 1 \leq m + n - 2,$$

since $\lfloor n/2 \rfloor \geq m + 1$. Thus every $n \geq 2m - 3$ is excluded from the tape-optimal region. The stated bound on $\alpha(m)$ follows directly from its definition, without a monotonicity conjecture for M . $\square$

Scope. This argument leaves the asymptotic upper boundary at ratio 2. It does not prove the equalities conjectured by Smith–Lang. The recurrence is the published Li–Sun–Zhang algorithm, and the finite inputs are published results. The contribution of this appendix is the explicit propagation and its carefully specified scope.

B Reproducing the finite proof

The arXiv source contains a single self-contained `main.tex` and an `anc/` directory of ancillary source code, data, and certificates. The document requires only standard L^AT_EX packages and compiles with `pdflatex main.tex`, run until references stabilize. No shell escape, external bibliography processor, or network access is needed. For the commands below, change into `anc/`. Python 3 and a compiler supporting C++17 suffice. `README.txt` gives the complete file map, and `SHA256SUMS` records the accepted files’ digests.

The quickest main-theorem check uses the supplied table:

```
python3 audit_arxiv/verify_main_theorem.py adversary/table_300.csv
```

It must report `PASS`, seven boundary checks, 138,622 translation inequalities, and 10,651 cone seeds. This checks the finite hypotheses; to recompute their table independently, run

```
g++ -O3 -std=c++17 dp_audit/fast_literal.cpp -o fast_literal
./fast_literal 300 300 > independent_table.csv
cmp adversary/table_300.csv independent_table.csv
python3 audit_arxiv/verify_main_theorem.py independent_table.csv
```

The `cmp` command must succeed without output. The independent full computation took about five minutes on the machine used for this work; this timing is descriptive and is not a complexity guarantee. The primary program and the smaller literal cross-check are also included.

The exact-instance proofs can be checked without rerunning their searches:

```

gzip -dk exact/certificate_6_22_safe.txt.gz
gzip -dk exact/certificate_6_25_safe.txt.gz
python3 audit_arxiv/check_state_validity.py
g++ -O3 -std=c++17 exact/verify_certificate.cpp -o verify_exact
./verify_exact exact/certificate_6_22_safe.txt 6 22
./verify_exact exact/certificate_6_25_safe.txt 6 25

```

The first Python check verifies the dimensions and integer contracts of every record. The final two runs must report `verified:true`, root values 21 and 22, and respectively 454,963 and 1,111,896 explicit records. Those runs took approximately 67 and 185 seconds, respectively, and cache several million elementary states in memory. The original verifier and supplementary Python check use assertions: do not compile with `-DNDEBUG`, run Python with `-O`, or set `PYTHONOPTIMIZE`. The standalone readable verifier additionally performs the record-contract checks itself and is described in the README.

The certificate file format is one record per line:

$$m \quad n \quad L \quad U \quad \ell_1 \cdots \ell_m \quad u_1 \cdots u_m.$$

Here L, U are asserted comparison bounds, while the endpoint vectors ℓ, u specify the state in Section 5. Records are canonical nontrivial components, with no duplicates. There are no embedded published merging values: missing states use exactly the elementary bounds proved in that section. The upper witness query is recovered by enumerating legal queries; a separate decision tree with one explicit leaf per full execution is unnecessary.

The two uncompressed certificate digests are, respectively,

```

7fff5828213ebcb112ba11f3685b8680
9baa852d62ed73632893b39d4f924412

1e14335f5c4d77021fd0f52df3650e6c
db360e11dd9096015260ad40ebd8450d

```

Concatenate the two lines within each digest. The ancillary manifest also identifies the compressed files, source programs, and finite check reports. Historical exploratory outputs and third-party papers are not part of the accepted arXiv proof data.

References

- [1] C. Christen. Improving the bounds on optimal merging. In *19th Annual Symposium on Foundations of Computer Science*, pp. 259–266. IEEE, 1978.
- [2] C. Christen. *On the optimality of the straight merging algorithm*. Unpublished manuscript, 1978. Bibliographic information and attribution as reported in [8, 9].
- [3] Wenceslas Fernández de la Vega, Sampath Kannan, and Miklós Santha. Two probabilistic results on merging. *SIAM Journal on Computing* 22(2):261–271, 1993. [doi:10.1137/0222019](https://doi.org/10.1137/0222019).
- [4] Frank K. Hwang. Optimal merging of 3 elements with n elements. *SIAM Journal on Computing* 9(2):298–320, 1980. [doi:10.1137/0209026](https://doi.org/10.1137/0209026).
- [5] Frank K. Hwang and Shen Lin. Optimal merging of 2 elements with n elements. *Acta Informatica* 1:145–158, 1971. [doi:10.1007/BF00289521](https://doi.org/10.1007/BF00289521).

- [6] Frank K. Hwang and Shen Lin. A simple algorithm for merging two disjoint linearly ordered sets. *SIAM Journal on Computing* 1(1):31–39, 1972. doi:10.1137/0201004.
- [7] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Second edition. Addison–Wesley, Reading, Massachusetts, 1998. Section 5.3.2, pp. 197–207. Author’s corrections: <https://cs.stanford.edu/~knuth/taocp.html>.
- [8] Qian Li, Xiaoming Sun, and Jialin Zhang. On the optimality of tape merge of two lists with similar size. In *27th International Symposium on Algorithms and Computation (ISAAC 2016)*, Leibniz International Proceedings in Informatics 64, article 51, pp. 51:1–51:17, 2016. doi:10.4230/LIPIcs.ISAAC.2016.51. Preprint: arXiv:1610.03266.
- [9] Qian Li, Xiaoming Sun, and Jialin Zhang. On the optimality of tape merge of two lists with similar size. *Algorithmica* 82(7):2107–2132, 2020. doi:10.1007/s00453-020-00690-x.
- [10] Nathan Linial. The information-theoretic bound is good for merging. *SIAM Journal on Computing* 13(4):795–801, 1984. doi:10.1137/0213049.
- [11] Glenn K. Manacher. Significant improvements to the Hwang–Lin merging algorithm. *Journal of the ACM* 26(3):434–440, 1979. doi:10.1145/322139.322144.
- [12] Paul E. Murphy. *A problem in optimal merging*. Technical Report DCS-TR-69, Department of Computer Science, Rutgers University, 1978. doi:10.7282/t3-24m2-c806.
- [13] Paul E. Murphy and Marvin C. Paull. Minimum comparison merging of sets of approximately equal size. *Information and Control* 42(1):87–96, 1979. doi:10.1016/S0019-9958(79)90186-4.
- [14] Jürgen Schulte Mönting. Merging of 4 or 5 elements with n elements. *Theoretical Computer Science* 14(1):19–37, 1981. doi:10.1016/0304-3975(81)90003-7.
- [15] Warren D. Smith and Kevin J. Lang. *Values of the merging function and algorithm design as a game*. Unpublished technical report, NEC Research Institute, 1994, with accompanying table and decision-tree drawings. <https://rangevoting.org/WarrenSmithPages/homepage/works.html> (author’s publication listing, with links to the report and table).
- [16] Paul K. Stockmeyer and F. Frances Yao. On the optimality of linear merge. *SIAM Journal on Computing* 9(1):85–90, 1980. doi:10.1137/0209006.
- [17] Mai Thanh and T. D. Bui. An improvement of the binary merge algorithm. *BIT* 22(4):454–462, 1982.